



SQL – DML

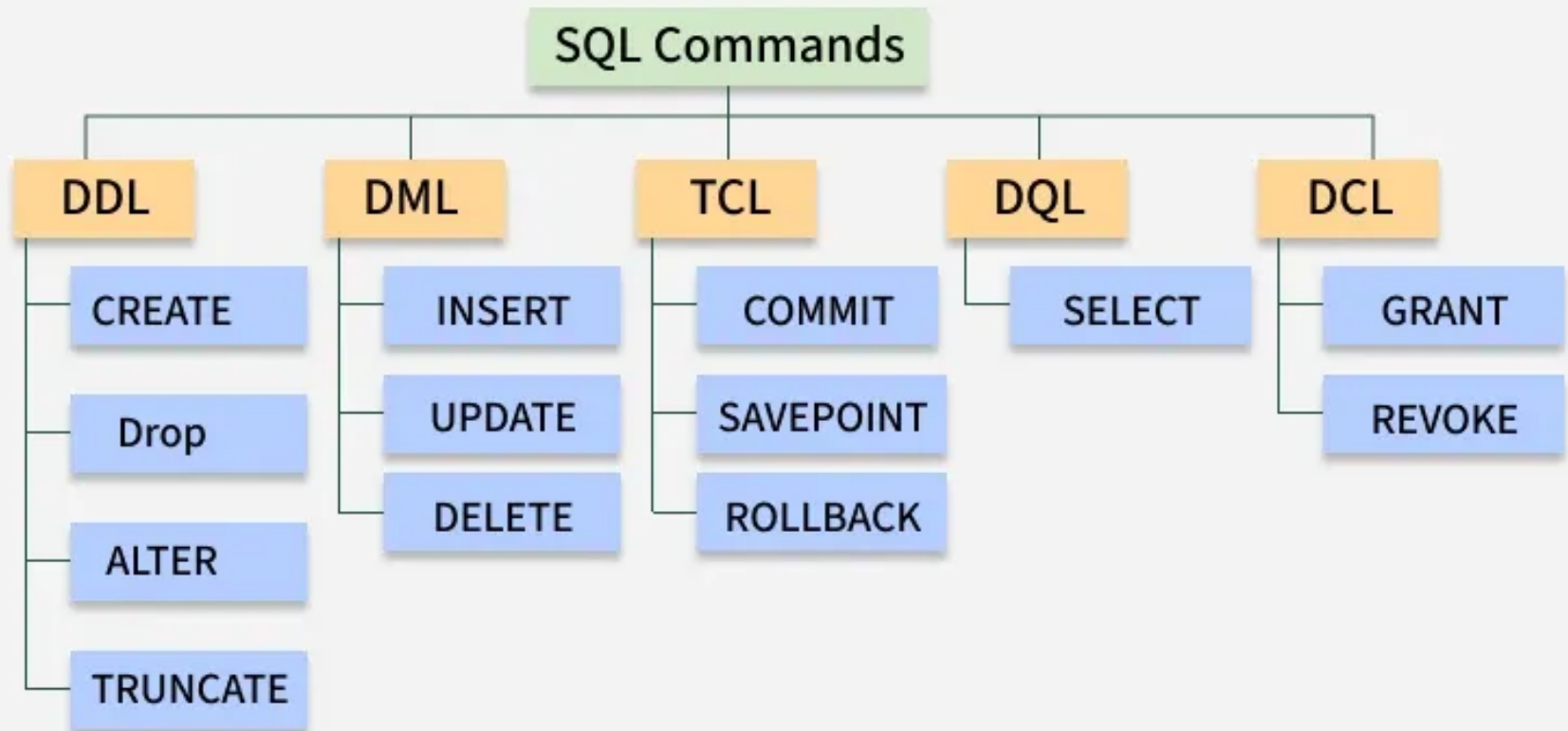
Agenda

- ▶ **Data Manipulation Language**
- ▶ **Examples & Exercises**



SQL - Data Manipulation Language

SQL Commands



SQL - DML

- ▶ It is a subset of SQL commands used to manipulate the data stored in database tables.
- ▶ With DML, you can insert new records, update existing ones, delete unwanted data or retrieve information.

SQL - DML

- ▶ **It includes:**
 - ▶ **INSERT**
 - ▶ **UPDATE**
 - ▶ **DELETE**

SQL – DML - INSERT

- ▶ **INSERT INTO** statement is used to add new rows to an existing table.
- ▶ It can insert values into all columns, specific columns, or even copy data from another table.
- ▶ This command is essential for populating databases with meaningful records such as customers, employees or students.

SQL – DML - INSERT

- ▶ **Inserting Data into All Columns**
 - ▶ This method is used when you want to insert data into all columns of a table without specifying column names.
 - ▶ We simply provide the values for each column, in the same order that the columns are defined in the table.

INSERT INTO table_name

VALUES (value1, value2, value3, ...);

SQL – DML - INSERT

- ▶ **Example- Student (ROLL_NO, NAME, ADDRESS, PHONE, AGE)**

INSERT INTO Student

VALUES (1, 'Liam', 'New York', 'xxxxxxxxxx', 18);

SQL – DML - INSERT

ROLL_NO	NAME	ADDRESS	PHONE	AGE
1	Liam	New York	xxxxxxxxxx	18
2	Sophia	Berlin	xxxxxxxxxx	18
3	Akira	Tokyo	xxxxxxxxxx	20
4	Carlos	Tokyo	xxxxxxxxxx	18
5	Isabella	Rome	xxxxxxxxxx	19

SQL – DML - INSERT

- ▶ **Inserting Data into Specific Columns**
 - ▶ In some cases, you might want to insert data into only certain columns, leaving the others empty or with default values. In such cases, we can specify the column names explicitly.

**INSERT INTO table_name (column1,
column2, column3, ...)**

VALUES (value1, value2, value3, ...);

SQL – DML - INSERT

- ▶ **Example- Student (ROLL_NO, NAME, ADDRESS, PHONE, AGE)**

```
INSERT INTO Student (ROLL_NO, NAME, AGE)  
VALUES (6, 'Hiroshi', 19);
```


SQL – DML - INSERT

ROLL_NO	NAME	ADDRESS	PHONE	AGE
1	Liam	New York	xxxxxxxxxx	18
2	Sophia	Berlin	xxxxxxxxxx	18
3	Akira	Tokyo	xxxxxxxxxx	20
4	Carlos	Tokyo	xxxxxxxxxx	18
5	Isabella	Rome	xxxxxxxxxx	19
6	Hiroshi	NULL	NULL	19

SQL – DML - INSERT

- ▶ **Inserting Multiple Rows at Once**
 - ▶ **Instead of running multiple INSERT INTO commands, you can insert multiple rows into a table in a single query. This is more efficient and reduces the number of database operations.**

SQL – DML - INSERT

**INSERT INTO table_name (column1,
column2, column3, ...)**

VALUES (value1, value2, value3, ...),

(value1, value2, value3, ...),

(value1, value2, value3, ...),

(value1, value2, value3, ...);

SQL – DML - INSERT

- ▶ **Example- Student (ROLL_NO, NAME, ADDRESS, PHONE, AGE)**

INSERT INTO Student

VALUES (1, 'Liam', 'New York', 'xxxxxxxxxx', 18),

(2, 'Sophia', 'Berlin', 'xxxxxxxxxxxx', 18),

(3, 'Akira', 'Tokyo', 'xxxxxxxxxxxx', 20),

(4, 'Carlos', 'Tokyo', 'xxxxxxxxxxxx', 18);

SQL – DML - INSERT

- ▶ **Inserting Data from One Table into Another Table**
 - ▶ We can also copy data from one table into another table using the **INSERT INTO SELECT** statement. This is very useful when we want to move or replicate data from one table to another without manually typing all the data.

```
INSERT INTO target_table  
SELECT * FROM source_table;
```


SQL – DML - INSERT

► Example

```
INSERT INTO Student  
SELECT * FROM OldStudent;
```

SQL – DML - UPDATE

- ▶ The UPDATE statement in SQL is used to modify existing records in a table without deleting and re-inserting them.
- ▶ It allows updating one or multiple columns, with or without conditions, to keep data accurate and consistent.

SQL – DML - UPDATE

- ▶ **Change specific column values in selected rows**
- ▶ **Apply targeted updates using WHERE**
- ▶ **Update single or multiple columns at once**
- ▶ **More efficient than deleting and re-inserting rows**
- ▶ **Maintains data integrity by modifying in place**

SQL – DML - UPDATE

- ▶ **Syntax:**

UPDATE table_name

SET column1 = value1, column2 = value2,...

WHERE condition;

- ▶ **Parameters**

- ▶ **table_name:** Name of the table you want to update.

- ▶ **SET:** The column(s) you want to update and their new values.

- ▶ **WHERE:** Filters the specific rows you want to update.

Example Table

CustomerID	CustomerName	LastName	Country	Age	Phone
1	Saad	Ali	Pakistan	23	9415536635
2	Fahad	Rahman	Malaysia	21	9812345678
3	Naveed	Ahmed	Sri Lanka	24	9123456789
4	Bilal	Hussain	Bangladesh	21	9876543210
5	Ahmed	Khan	Indonesia	22	7012345678

SQL – DML - UPDATE

- ▶ Updating single column
- ▶ Syntax

UPDATE Customer

SET CustomerName = 'Arsalan'

WHERE Age = 22;

Result

CustomerID	CustomerName	LastName	Country	Age	Phone
1	Saad	Ali	Pakistan	23	9415536635
2	Fahad	Rahman	Malaysia	21	9812345678
3	Naveed	Ahmed	Sri Lanka	24	9123456789
4	Bilal	Hussain	Bangladesh	21	9876543210
5	Arsalan	Khan	Indonesia	22	7012345678

SQL – DML - UPDATE

- ▶ **Updating Multiple Columns using UPDATE Statement**

- ▶ **Syntax**

UPDATE Customer

SET CustomerName = 'Ahsan',

LastName = 'Jawed'

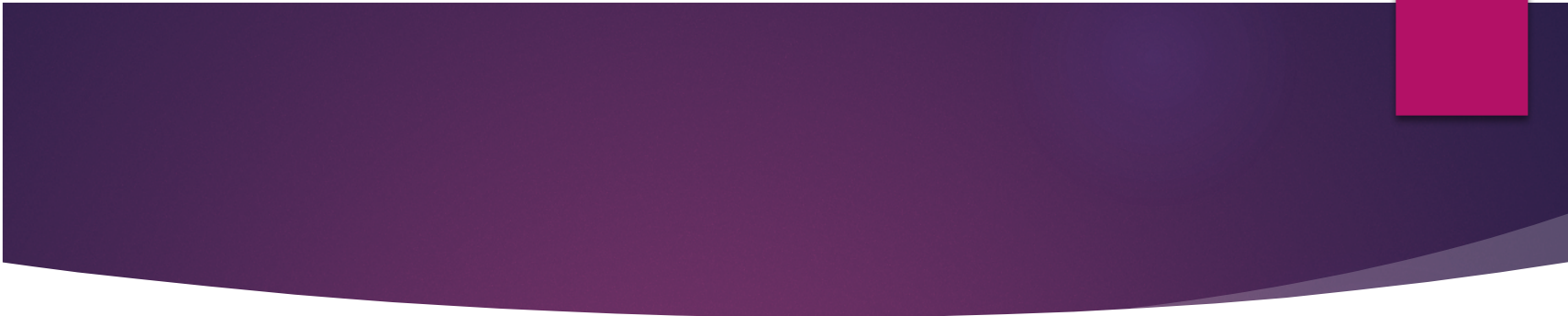
WHERE Country = 'Pakistan';

Result

CustomerID	CustomerName	LastName	Country	Age	Phone
1	Ahsan	Jawed	Pakistan	23	9415536635
2	Fahad	Rahman	Malaysia	21	9812345678
3	Naveed	Ahmed	Sri Lanka	24	9123456789
4	Bilal	Hussain	Bangladesh	21	9876543210
5	Arsalan	Khan	Indonesia	22	7012345678

SQL – DML - UPDATE

- ▶ **Omitting WHERE Clause in UPDATE Statement Syntax**
UPDATE Customer
SET Age = 22;



CustomerID	CustomerName	LastName	Country	Age	Phone
1	Ahsan	Jawed	Pakistan	22	9415536635
2	Fahad	Rahman	Malaysia	22	9812345678
3	Naveed	Ahmed	Sri Lanka	22	9123456789
4	Bilal	Hussain	Bangladesh	22	9876543210
5	Arsalan	Khan	Indonesia	22	7012345678

SQL – DML - DELETE

- ▶ The SQL DELETE statement is used to remove specific rows from a table while keeping the table structure intact. It is different from DROP, which deletes the entire table.
- ▶ Removes rows based on conditions.
- ▶ Retains table schema, constraints, and indexes.
- ▶ Can delete a single row or all rows.
- ▶ Useful for cleaning or managing datasets.

SQL – DML - DELETE

► Syntax

```
DELETE FROM table_name  
WHERE Condition;
```

► Example

```
Delete from Customers  
Where CustomerID = 5;
```

Result

CustomerID	CustomerName	LastName	Country	Age	Phone
1	Ahsan	Jawed	Pakistan	22	9415536635
2	Fahad	Rahman	Malaysia	22	9812345678
3	Naveed	Ahmed	Sri Lanka	22	9123456789
4	Bilal	Hussain	Bangladesh	22	9876543210

SQL – DML - DELETE

- ▶ **Deleting Multiple Records**

- ▶ **Example**

Delete from Customers

Where CustomerID > 2;

CustomerID	CustomerName	LastName	Country	Age	Phone
1	Ahsan	Jawed	Pakistan	22	9415536635
2	Fahad	Rahman	Malaysia	22	9812345678

SQL – DML - DELETE

- ▶ **Deleting All Records**

- ▶ **Example**

DELETE FROM Customers;

Or

DELETE * FROM Customers;



THANK YOU